TP3 — Robotique : Résolution d'un labyrinthe par Q-learning

Jérémy Doublet et Mohamadou Sall

22 avril 2026

1 Introduction

L'objectif de ce TP est de mettre en œuvre l'algorithme de Q-learning afin de résoudre un problème de navigation dans un labyrinthe. Le Q-learning est une méthode d'apprentissage par renforcement permettant à un agent d'apprendre une politique optimale sans connaissance préalable de l'environnement.

2 Modélisation du problème

Le labyrinthe est représenté sous forme d'une matrice discrète dans laquelle :
— 0 représente une case libre,
— 1 représente un obstacle,
— S correspond à la position de départ,
— G correspond à la position objectif.
Chaque état correspond à une position dans la grille :

$$s = (i, j) \tag{1}$$

Les actions possibles sont :

$$a \in \uparrow, \downarrow, \rightarrow, \leftarrow \tag{2}$$

3 Fonction de récompense

La fonction de récompense est définie comme suit :
— $+100$ si l'objectif est atteint,
— -100 en cas de collision avec un obstacle,
— -1 pour chaque déplacement.
Cette définition permet d'encourager l'agent à atteindre l'objectif en minimisant le nombre de déplacements.

4 Algorithme Q-learning

La Q-table est initialisée à zéro :

$$Q(s, a) = 0 \tag{3}$$

Une stratégie ϵ-greedy est utilisée :
— exploration avec probabilité ϵ,
— exploitation avec probabilité $1 - \epsilon$.

Le paramètre ϵ évolue selon :

$$\epsilon \leftarrow \max(\epsilon_{min}, \beta \cdot \epsilon) \tag{4}$$

La mise à jour de la Q-table est donnée par :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right) \tag{5}$$

5 Implémentation

L'algorithme est implémenté en Python en suivant le pseudo-code fourni. À chaque étape, l'agent choisit une action, observe la récompense et met à jour la Q-table.

6 Résultats

La convergence de l'algorithme est évaluée à l'aide de la récompense cumulée par épisode.

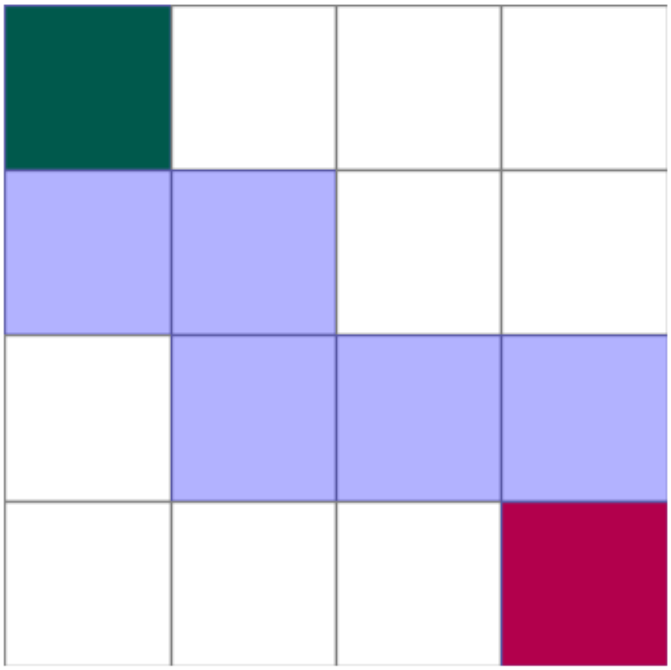

FIGURE 1 – Labyrinthe du Q-learning

Une visualisation du labyrinthe permet également d'observer le chemin optimal appris par l'agent.

7 Conclusion

Le Q-learning permet d'apprendre une politique optimale sans connaissance préalable de l'environnement. L'agent apprend progressivement à éviter les obstacles et à atteindre l'objectif de manière efficace.

Cependant, cet algorithme peut être plus lent que des méthodes déterministes comme Dijkstra ou A*, notamment dans des environnements simples et statiques.

8 Travaux et Questions

1 — Implémenter Q-learning et montrer la convergence

L'algorithme de Q-learning a été implémenté conformément à la règle de mise à jour de Bellman :

[Q(s,a) $\leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$]

La convergence est observée à l'aide de la récompense cumulée par épisode. Une courbe de convergence est tracée afin de visualiser l'évolution de l'apprentissage :

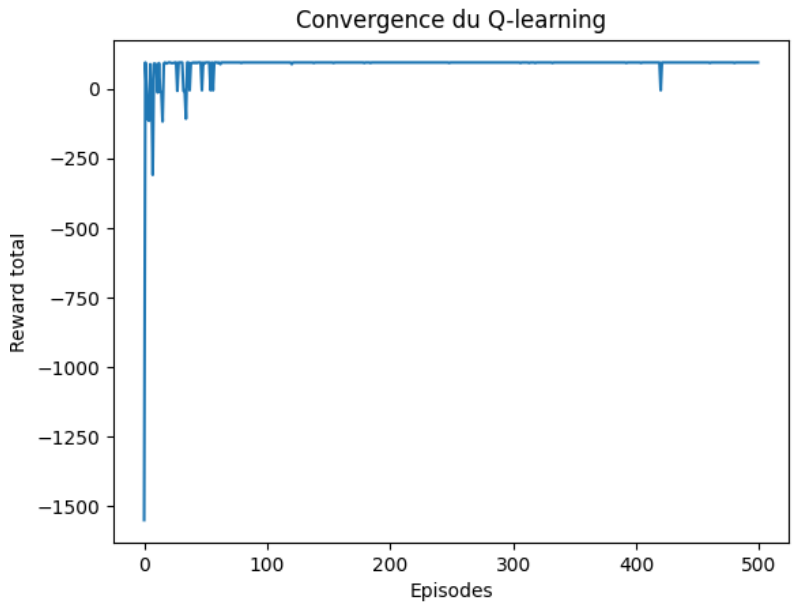

FIGURE 2 – Courbe de convergence du Q-learning

On observe une phase d'exploration initiale suivie d'une stabilisation progressive, signe de convergence vers une politique optimale.

2 — Test sur graphes de grande taille

Le Q-learning a été testé sur plusieurs types de graphes issus de la bibliothèque NetworkX :

— Random geometric graph : [G = $\texttt{nx.random}_{g}eometric_{g}raph(2000, 0.05)$]

“‘

— Graphe d'Erdős–Rényi :

$$G = \texttt{nx.erdos_renyi_graph}(1000, 0.01)$$

— Graphe de Barabási–Albert (scale-free) :

$$G = \texttt{nx.barabasi_albert_graph}(1000, 5)$$

“‘

Les résultats montrent que la convergence dépend fortement de la structure du graphe. Les graphes scale-free (Barabási–Albert) convergent généralement plus rapidement grâce à la présence de hubs facilitant l'exploration.

3 — Différence entre Q-learning et Dijkstra / A*

Dijkstra et A* sont des algorithmes de recherche de plus court chemin nécessitant une connaissance complète du graphe.

Q-learning, en revanche, est un algorithme d'apprentissage par renforcement qui :

— ne nécessite pas de connaissance préalable de l'environnement,

— apprend par essais et erreurs,
— converge progressivement vers une politique optimale.

Dijkstra garantit une solution optimale immédiatement, tandis que Q-learning nécessite une phase d'apprentissage mais est plus adapté aux environnements dynamiques.

4 — Pourquoi une récompense négative à chaque déplacement ?

Une pénalité à chaque déplacement ($r = -1$) est utilisée afin de :
— encourager l'agent à trouver le chemin le plus court,
— éviter les comportements de stagnation,
— accélérer la convergence vers une solution efficace.

Sans cette pénalité, l'agent pourrait errer sans contrainte de durée.

5 — Rôle du paramètre ε

Le paramètre ε contrôle le compromis exploration/exploitation :
— ε élevé : exploration (actions aléatoires),
— ε faible : exploitation (meilleure action connue).

Il permet d'éviter les optima locaux et de découvrir progressivement la structure de l'environnement.

6 — Cas $\varepsilon = 0$ et $\varepsilon = 1$
— $\varepsilon = 0$: exploitation pure $\Rightarrow$ risque de rester bloqué dans une solution sous-optimale.
— $\varepsilon = 1$: exploration pure $\Rightarrow$ apprentissage inefficace, comportement aléatoire.

Un compromis entre exploration et exploitation est donc essentiel.

7 — Vérification de la convergence

La convergence peut être vérifiée par :
— la stabilisation de la récompense cumulée,
— la stabilisation des valeurs de la Q-table,
— la constance du chemin extrait.

On peut également mesurer : $[\ \Delta Q_{\max} \to 0] pourconfirmerlaconvergence.$

8 — Q-learning trouve-t-il toujours le chemin optimal ?

Théoriquement, Q-learning converge vers la politique optimale sous certaines conditions (exploration suffisante et apprentissage décroissant).

En pratique, il peut converger vers une solution sous-optimale si :
— le nombre d'épisodes est insuffisant,
— l'exploration est trop faible,
— l'environnement est trop complexe.

9 — Cas où Q-learning est moins performant que Dijkstra

Q-learning est moins performant lorsque :
— le graphe est connu et statique,
— une solution exacte immédiate est requise,
— l'espace d'état est très grand sans fonction d'approximation.

Dans ces cas, Dijkstra ou A* sont plus efficaces et garantissent l'optimalité immédiate.

10 — Adaptation aux environnements dynamiques

Q-learning est adapté aux environnements dynamiques car :
— il ne nécessite pas de modèle de l'environnement,
— il apprend directement à partir de l'expérience,
— il s'adapte aux changements via les mises à jour continues de la Q-table.

Ainsi, il est particulièrement utilisé en robotique et en intelligence artificielle pour des environnements évolutifs ou inconnus.